EXPERIMENT - 74

Question 1

1. Problem Statement

Create an enumeration for months and print how many days each month has.
Input: FEB
Output: 28 or 29 days

2. Algorithm

Step 1: START
Step 2: Declare enum Month with JAN = 1 up to DEC = 12.
Step 3: Read the month name and change its letters to capital letters.
Step 4: Find the month in the list. If it is not there, display Invalid month and stop.
Step 5: If the month is FEB display 28 or 29 days. If it is APR, JUN, SEP or NOV display 30 days. Otherwise display 31 days.
Step 6: STOP

3. Test Case Table

#	Input	Expected Output	Actual Output	Result
1	FEB	28 or 29 days	28 or 29 days	PASS
2	APR	30 days	30 days	PASS
3	DEC	31 days	31 days	PASS
4	feb	28 or 29 days	28 or 29 days	PASS
5	XYZ	Invalid month	Invalid month	PASS

4. Source Code

month74.c

```
/* Experiment 74, Question 1: enumeration for months and their days. */
#include <stdio.h>
#include <string.h>
#include <ctype.h>

enum Month { JAN = 1, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC };

int main(void)
{
    const char *names[] = {"JAN", "FEB", "MAR", "APR", "MAY", "JUN",
                           "JUL", "AUG", "SEP", "OCT", "NOV", "DEC"};
    char input[20];
    enum Month month;
    int i;

    printf("Enter the month (JAN to DEC): ");
    scanf("%19s", input);
    for (i = 0; input[i] != '\0'; i++)
        input[i] = toupper((unsigned char)input[i]);

    for (month = JAN; month <= DEC; month++)
        if (strcmp(input, names[month - 1]) == 0)
            break;
    if (month > DEC) {
        printf("Invalid month\n");
        return 1;
    }

    switch (month) {
    case FEB:
        printf("28 or 29 days\n");
        break;
    case APR: case JUN: case SEP: case NOV:
        printf("30 days\n");
        break;
    default:
        printf("31 days\n");
    }
    return 0;
}
```

Question 2

1. Problem Statement

Define an enumeration with SUCCESS, FAILURE, and TIMEOUT, and print messages accordingly.
Input: FAILURE
Output: Operation Failed

2. Algorithm

Step 1: START
Step 2: Declare enum Status with SUCCESS, FAILURE and TIMEOUT.
Step 3: Read the status and change its letters to capital letters.
Step 4: If it is SUCCESS, FAILURE or TIMEOUT, set status to that value. Otherwise display Invalid status and stop.
Step 5: If status is SUCCESS display Operation Successful, if FAILURE display Operation Failed, and if TIMEOUT display Operation Timed Out.
Step 6: STOP

3. Test Case Table

#	Input	Expected Output	Actual Output	Result
1	FAILURE	Operation Failed	Operation Failed	**PASS**
2	SUCCESS	Operation Successful	Operation Successful	**PASS**
3	TIMEOUT	Operation Timed Out	Operation Timed Out	**PASS**
4	failure	Operation Failed	Operation Failed	**PASS**
5	ERROR	Invalid status	Invalid status	**PASS**

4. Source Code

status74.c

```
/* Experiment 74, Question 2: enumeration for the result of an operation. */
#include <stdio.h>
#include <string.h>
#include <ctype.h>

enum Status { SUCCESS, FAILURE, TIMEOUT };

int main(void)
{
    char input[20];
    enum Status status;
    int i;

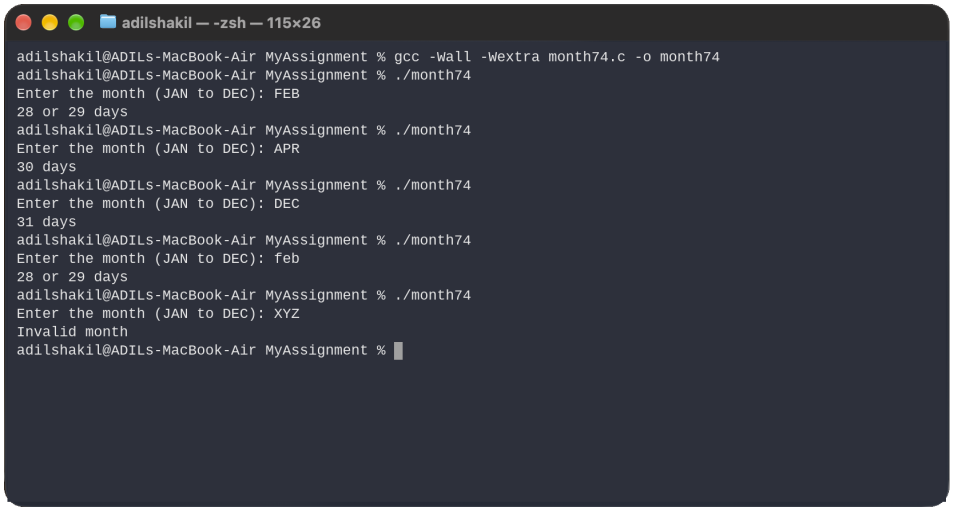
    printf("Enter the status (SUCCESS, FAILURE or TIMEOUT): ");
    scanf("%19s", input);
    for (i = 0; input[i] != '\0'; i++)
        input[i] = toupper((unsigned char)input[i]);

    if (strcmp(input, "SUCCESS") == 0)
        status = SUCCESS;
    else if (strcmp(input, "FAILURE") == 0)
        status = FAILURE;
    else if (strcmp(input, "TIMEOUT") == 0)
        status = TIMEOUT;
    else {
        printf("Invalid status\n");
        return 1;
    }

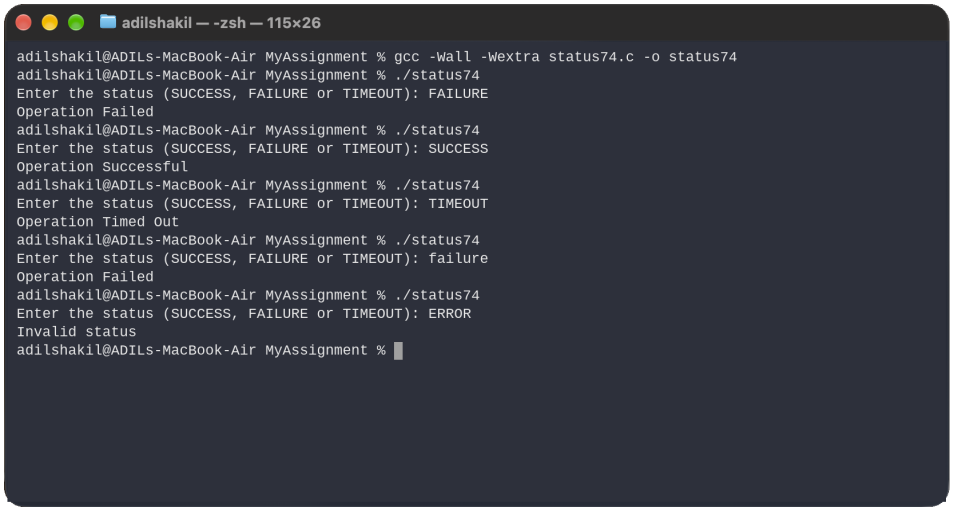
    switch (status) {
    case SUCCESS:
        printf("Operation Successful\n");
        break;
    case FAILURE:
        printf("Operation Failed\n");
        break;
    case TIMEOUT:
        printf("Operation Timed Out\n");
        break;
    }
    return 0;
}
```

5. Compilation and Output

Question 1: compiled with `gcc -Wall -Wextra month74.c -o month74`



Question 2: compiled with `gcc -Wall -Wextra status74.c -o status74`



Both programs built clean, and the first test of each is the example from the question.